

Solution Requirements

Date 29/09/2026

Team ID SWTID-2026-1375

Project Name FitBuddy – AI Fitness Plan Generator using Gemini Models

Maximum Marks 4 Marks

Define Functional and Non-Functional Requirements:

Create a comprehensive list of requirements to define exactly what your solution will do and how well it will perform.

Functional Requirements (FR) define specific behaviors, functions, or features of the system (What the system should do).

Non-Functional Requirements (NFR) specify the criteria that judge the operation of a system, rather than specific behaviors (How the system should be).

Step 1: Functional Requirements (FR)

S.No	Paragraph('caseSensitive': 1 'encoding': 'utf8' 'text': 'Requirement Category' 'frags': [ParaFrag(__tag__='b', bold=1, fontName='Helv etica-Bold', fontSize=7.2, greek=0, italic=0, link=[], rise=0, text='Requirement Category', textColor=Co lor(0,0,0,1), us_lines=[])] 'style': 'bulletText': None 'debug': 0) #Paragraph	Paragraph('caseSensitive': 1 'encoding': 'utf8' 'text': ' Requirement Description ' 'frags': [ParaFrag(__tag__='b', bold=1, fontName='Helvetica-Bold', fontSize=7.2, greek=0, italic=0, link=[], rise=0, text='Requirement Description', textColor=Color(0,0,0,1), us_lines=[])] 'style': 'bulletText': None 'debug': 0) #Paragraph	Paragraph('caseSensitive': 1 'encoding': 'utf8' 'text': ' Priority ' 'frags': [ParaFrag(__tag__='b', bold=1, fontName='Helvetic a-Bold', fontSize=7.2, greek=0, italic=0, link=[], rise=0, text='Priority', textC olor=Color(0,0,0,1), us_lines=[])] 'style': 'bulletText': None 'debug': 0) #Paragraph
1	Authentication	Users provide identifying details such as username and user ID through the FitBuddy input form before generating a plan.	Medium
2	Authorization levels	Admin/coach can access the View All Users dashboard to view registered users and their original and updated workout plans.	High
3	External interfaces	The application connects to Google Gemini APIs for workout-plan generation, feedback-based updates, and nutrition/recovery tips.	High
4	Transactions processing	FastAPI receives user form data, validates it with Pydantic schemas, calls the required Gemini function, stores user/plan data in SQLite, and returns results through Jinja2 templates.	High
5	Reporting	The admin dashboard displays username, age, weight, goal, intensity, and original/updated workout plans in a structured table.	Medium
6	Business rules, etc.	The system generates a customized 7-day plan from user goal and workout intensity; feedback such as more cardio, yoga, or rest days is used to update the plan.	High
7	Compliance to laws or regulations	The project document does not specify a particular legal or regulatory standard. User information should be handled according to the application's privacy and data-handling requirements.	Medium
8	Other	The system provides a nutrition or recovery tip and supports web pages for input, personalized results, feedback, and admin viewing.	Medium

Step 2: Non-Functional Requirements (NFR)

S.No	Paragraph('caseSensitive': 1 'encoding': 'utf8' 'text': 'NFR Category' 'frags': [ParaFrag(__tag__='b', bold=1, fontName='Helvetica-Bold', fontSize=7.2, greek=0, italic=0, link=[], rise=0, text='NFR Category', textColor=Color(0,0,0,1), us_lines=[]) 'style': 'bulletText': None 'debug': 0) #Paragraph	Paragraph('caseSensitive': 1 'encoding': 'utf8' 'text': 'Requirement Description' 'frags': [ParaFrag(__tag__='b', bold=1, fontName='Helvetica-Bold', fontSize=7.2, greek=0, italic=0, link=[], rise=0, text='Requirement Description', textColor=Color(0,0,0,1), us_lines=[]) 'style': 'bulletText': None 'debug': 0) #Paragraph	Paragraph('caseSensitive': 1 'encoding': 'utf8' 'text': 'Target Metric / Acceptance Criteria' 'frags': [ParaFrag(__tag__='b', bold=1, fontName='Helvetica-Bold', fontSize=7.2, greek=0, italic=0, link=[], rise=0, text='Target Metric / Acceptance Criteria', textColor=Color(0,0,0,1), us_lines=[]) 'style': 'bulletText': None 'debug': 0) #Paragraph
1	Performance & Speed	Workout generation and nutrition-tip requests should be processed efficiently through FastAPI and Gemini APIs.	Normal user request should return a result without unnecessary delay; API/server remains responsive.
2	Scalability	The modular FastAPI architecture should allow growth in users, plans, and AI requests.	Architecture supports adding more users and plans without changing the main workflow.
3	Security & Data Privacy	User details and generated plans are stored in SQLite; the Gemini API key is configured through an environment variable.	API key is not hard-coded in application pages; stored user/plan data is protected from unauthorized access.
4	Reliability & Availability	The application should run reliably on the local FastAPI/Uvicorn server and preserve original and updated plans.	Application starts successfully; original and updated plan records remain available in the database.
5	Usability & Accessibility	The interface should provide clear input fields, readable result blocks, responsive layouts, and simple navigation.	Users can enter required details, generate a plan, view results, submit feedback, and see confirmation without confusion.
6	Other	Maintainability and portability through modular FastAPI routes, Jinja2 templates, Gemini integration modules, and SQLAlchemy.	Core features remain separated into routes, templates, AI functions, and database functions for easier maintenance.

Source basis: FitBuddy project document. The requirements above preserve the project's stated architecture, features, routes, Gemini integration, SQLite storage, and user/admin workflows. ■filecite■turn3file0■L97-L114■